

Workspace-Dokumentation: gubot_one

Automatisch generiert aus dem Quellcode-Analyse-Durchlauf.

Paket: `gubot_one` | ROS2 (Humble/Iron) | Stand: 2025

Inhaltsverzeichnis

- 1. [Überblick & Architektur](#)
- 2. [description/ – Roboterbeschreibung \(URDF/Xacro\)](#)
- 3. [launch/ – Launch-Dateien](#)
- 4. [Abhängigkeitsmatrix](#)

Überblick & Architektur

Der `gubot_one` ist ein differenzialgetriebener Roboter mit:

- **Antrieb:** 2 angetriebene Räder (links/rechts) + 1 Stützrad
- **Sensorik:** IMU, USB-Kamera (optional), RPLidar (optional)
- **Sonderausstattung:** Nerf-Launcher (Arduino/Pico gesteuert)
- **Steuerung:** `ros2_control` mit `diffdrive_arduino` -Plugin (Hardware) / Ignition/Gazebo-Plugin (Simulation)

```
robot.urdf.xacro
├─ robot_core.xacro      (Chassis, Räder, IMU)
├─ inertial_macros.xacro (Trägheitsmoment-Helfer)
├─ ros2_control.xacro    (Ignition Sim + echte Hardware)
│   └─ ODER
│       └─ gz_classic_ros2_control.xacro (Gazebo Classic)
│           └─ ODER
│               └─ gazebo_control.xacro (Legacy Diff-Drive Plugin)
├─ camera.xacro          (RGB-Kamera)
├─ lidar.xacro            (auskommentiert)
├─ face.xacro             (Gesichts-Visualisierung)
└─ nerf_launcher.xacro    (Nerf-Launcher mount)
```

description/ – Roboterbeschreibung (URDF/Xacro)

`robot.urdf.xacro`

Pfad: `description/robot.urdf.xacro`

Was es ist: Die Haupt-URDF-Einstiegsdatei. Sie wird von nahezu allen Launch-Dateien über `xacro` verarbeitet und steuert per Argumente, welche Subsysteme eingebunden werden.

Argumente:

Argument	Standard	Bedeutung
<code>use_ros2_control</code>	<code>true</code>	Aktiviert <code>ros2_control</code> statt Legacy-Plugin

sim_mode	false	Schaltet zwischen Hardware und Simulation
integrated_mode	true	Nerf-Launcher + Basis zusammen starten
use_nerf_hardware	true	Nerf-Hardware-Interface einbinden
nerf_port	/dev/serial/...Leonardo	Serieller Port des Arduino
use_gazebo_classic	false	Gazebo Classic statt Ignition Gazebo

Logik:

```

use_ros2_control=true + use_gazebo_classic=true → gz_classic_ros2_control.xacro
use_ros2_control=true + use_gazebo_classic=false → ros2_control.xacro
use_ros2_control=false → gazebo_control.xacro (Legacy)

```

Immer eingebunden: robot_core.xacro , camera.xacro , face.xacro , nerf_launcher.xacro

Auskommentiert: lidar.xacro , depth_camera.xacro

Verwendet von:

- rsp.launch.py (via xacro -Command zur Laufzeit)
- launch_robot.launch.py (direkt als robot_description für Controller Manager)
- launch_sim.launch.py
- gz_classic_launch_sim.launch.py

robot_core.xacro

Pfad: description/robot_core.xacro

Was es ist: Definiert die komplette physische Struktur des Roboters: Chassis, beide Antriebsräder, Stützrad und IMU-Link. Enthält alle Maße, Massen, Materialien und Gazebo-Sensor-Definitionen.

Eingebundene Dateien: inertial_macros.xacro

Definierte Links & Joints:

Link	Typ	Beschreibung
base_link	leer	Referenzpunkt für alle anderen Links
base_footprint	leer	Projektion auf den Boden (für Navigation)
chassis	Box 335×265×138 mm	Orangefarbener Hauptkörper
imu_link	Box 20×20×20 mm	IMU-Sensor, 5 cm über Chassis
left_wheel	Zylinder r=33mm	Linkes Antriebsrad
right_wheel	Zylinder r=33mm	Rechtes Antriebsrad
caster_wheel	Kugel r=10mm	Passives Stützrad vorne

IMU-Konfiguration (in Gazebo):

- Update-Rate: 50 Hz

- Rauschen konfiguriert für Winkelgeschwindigkeit und Beschleunigung
- Gazebo Classic: `libgazebo_ros_imu_sensor.so` , Topic: `imu/data`
- Ignition: Topic: `imu_sensor/imu_data`

Räder:

- `left_wheel_joint` / `right_wheel_joint` : `continuous` -Joints (drehbar ohne Limit)
- Kollisionsgeometrie: Kugel (stabiler in Simulation als Zylinder)
- Caster: $\mu_1/\mu_2 = 0.001$ (reibungssarm für realistisches Verhalten)

Verwendet von: `robot.urdf.xacro` (direkt via `<xacro:include>`)

`inertial_macros.xacro`

Pfad: `description/inertial_macros.xacro`

Was es ist: Bibliothek mit drei Xacro-Makros zur Berechnung physikalisch korrekter Trägheitsmomente. Wird von fast allen anderen Xacro-Dateien genutzt.

Makros:

Makro	Formel	Verwendet für
<code>inertial_sphere</code>	$I = (2/5)mr^2$	Caster Wheel
<code>inertial_box</code>	Standard Quader-Formel	Chassis, IMU
<code>inertial_cylinder</code>	Standard Zylinder-Formel	Antriebsräder

Verwendet von: `robot_core.xacro` , `lidar.xacro` (und implizit überall wo `robot_core.xacro` eingebunden ist)

`ros2_control.xacro`

Pfad: `description/ros2_control.xacro`

Was es ist: `ros2_control`-Konfiguration für **Ignition Gazebo** (Simulation) und **echte Hardware**. Definiert welche Controller-Plugins geladen werden und welche Joints/Sensoren sie steuern.

Hardware-Modus (`sim_mode=false` → **echte Hardware**):

- Plugin: `diffdrive_arduino/DiffDriveArduinoHardware`
- Gerät: Raspberry Pi Pico via USB (`usb-Raspberry_Pi_Pico...`)
- Baud-Rate: 115200, Loop-Rate: 30 Hz
- Encoder: 3436 Counts/Umdrehung
- Joints: `left_wheel_joint` , `right_wheel_joint` (velocity command)

Nerf-Hardware (`use_nerf_hardware=true`):

- Plugin: `nerf_launch_system/NerfSystem`
- Port: Arduino Leonardo (konfigurierbar über `nerf_port`)
- Joints: `trigger_joint` , `dart_pusher_joint` , `flywheel_left_joint` , `flywheel_right_joint` , `system_arwing_joint`

Simulations-Modus (`sim_mode=true`):

- Plugin: `ign_ros2_control/IgnitionSystem`
- Alle Wheel-Joints + alle Nerf-Joints + IMU-Sensor
- `trigger_joint` startet bei Position 6.28 rad (Sicherheitsstellung)

Gazebo-Plugin: `ign_ros2_control-system` lädt `my_controllers.yaml` + `gaz_ros2_ctl_use_sim.yaml`

Verwendet von: `robot.urdf.xacro` (wenn `use_ros2_control=true` und `use_gazebo_classic=false`)

`gz_classic_ros2_control.xacro`

Pfad: `description/gz_classic_ros2_control.xacro`

Was es ist: Identisch zu `ros2_control.xacro`, aber für **Gazebo Classic** angepasst. Der einzige Unterschied liegt im Simulations-Plugin (`GazeboSystem` statt `IgnitionSystem`) und dem geladenen Gazebo-Plugin (`libgazebo_ros2_control.so` statt `ign_ros2_control-system`).

Unterschied zu `ros2_control.xacro` :

Eigenschaft	<code>ros2_control.xacro</code>	<code>gz_classic_ros2_control.xacro</code>
Sim-Plugin	<code>ign_ros2_control/IgnitionSystem</code>	<code>gazebo_ros2_control/GazeboSystem</code>
Gazebo-Plugin	<code>ign_ros2_control-system</code>	<code>libgazebo_ros2_control.so</code>

Verwendet von: `robot.urdf.xacro` (wenn `use_ros2_control=true` und `use_gazebo_classic=true`)

`gazebo_control.xacro`

Pfad: `description/gazebo_control.xacro`

Was es ist: Legacy-Fallback. Nutzt das ältere `libgazebo_ros_diff_drive.so` -Plugin **ohne** `ros2_control`. Kein separates Controller-Framework – Gazebo übernimmt direkt Odometrie und Steuerung.

Parameter:

- Radabstand: 297 mm
- Raddurchmesser: 66 mm
- Publiziert: Odometrie-Topic, TF `odom` → `base_link`, Rad-TF

Verwendet von: `robot.urdf.xacro` (wenn `use_ros2_control=false`)

Hinweis: In der aktuellen Konfiguration standardmäßig **nicht aktiv** (`use_ros2_control` default: `true`)

`camera.xacro`

Pfad: `description/camera.xacro`

Was es ist: Definiert den RGB-Kamera-Link und das Gazebo-Sensor-Plugin. Die Kamera ist vorne am Chassis montiert, leicht nach unten geneigt.

Physische Montage:

- Position: `xyz="0.276 0 0.181"` (vorne, oben am Chassis)
- Neigung: 0.18 rad ($\approx 10^\circ$) nach unten

Links:

- `camera_link` : Visuelle Box + Halterungsstab (schwarz)
- `camera_link_optical` : Optischer Frame (rotiert um -90° in X und Z für REP-103-konforme Ausrichtung)

Sensor-Konfiguration:

- Auflösung: 640×480, RGB
- Sichtfeld: 1.089 rad (≈62°)
- Clip: 5 cm bis 8 m
- Update-Rate: 10 Hz
- Gazebo Classic: Plugin `libgazebo_ros_camera.so`
- Ignition: Topic `/camera/image_raw`

Verwendet von: `robot.urdf.xacro` (immer eingebunden)

Korrespondierendes Topic (echte Hardware): `/camera/image_raw` via `camera.launch.py` oder `real_camera.launch.py`

`lidar.xacro`

Pfad: `description/lidar.xacro`

Was es ist: Simulierter LIDAR-Sensor (RPLidar-ähnlich). Aktuell **auskommentiert** in `robot.urdf.xacro` – der echte RPLidar wird über `rplidar.launch.py` als eigenständiger Node gestartet.

Physische Montage:

- Position: `xyz="0.122 0 0.212"` (oben auf dem Chassis)
- Zylinder: r=50 mm, h=40 mm (schwarz)

Sensor-Konfiguration:

- 360°-Scan, 360 Samples
- Range: 0.3 m – 12 m
- Gazebo Classic: Ray-Sensor mit `libgazebo_ros_ray_sensor.so`, Topic `/scan`
- Ignition: GPU-Lidar, Topic `scan`

Verwendet von: `robot.urdf.xacro` (auskommentiert – derzeit nicht aktiv)

`depth_camera.xacro`

Pfad: `description/depth_camera.xacro`

Was es ist: Tiefenkamera-Definition (z. B. Intel RealSense-ähnlich) für Gazebo Classic. Aktuell **auskommentiert** in `robot.urdf.xacro`.

Physische Montage:

- Position: `xyz="0.305 0 0.08"` (vorne, niedriger als RGB-Kamera)
- Visualisierung in Rot (zur Unterscheidung)

Sensor-Konfiguration:

- Typ: `depth` (Tiefenkamera)
- Auflösung: 640×480, BGR
- Clip: 5 cm – 8 m
- Plugin: `libgazebo_ros_camera.so` mit min/max depth

Verwendet von: `robot.urdf.xacro` (auskommentiert – derzeit nicht aktiv)

face.xacro

Pfad: description/face.xacro

Was es ist: Rein visueller Link – simuliert ein „Gesicht“ am vorderen Ende des Chassis. Zwei kleine Zylinder als „Augen“ (schwarz) und ein großer flacher Zylinder als „Mund“ (rot).

Physische Montage:

- Position: Am vorderen Ende des Chassis (`${chassis_length}`)
- Höhe: Mitte des Chassis

Verwendet von: robot.urdf.xacro (immer eingebunden)

Hinweis: Rein dekorativ, kein Sensor oder Aktuator.

nerf_launcher.xacro

Pfad: description/nerf_launcher.xacro

Was es ist: Mount-Definition für den Nerf-Launcher. Bindet die URDF des externen Pakets `nerf_launch_system` ein und verbindet dessen `bottom_plate` -Link mit dem Chassis.

Montage:

- Parent: `chassis`
- Child: `bottom_plate` (aus `nerf_launch_system`)
- Position: `xyz="0.155 0 0.165"` (zentral oben auf dem Chassis)
- Rotation: 90° um Z-Achse (`${pi/2}`)
- `preserveFixedJoint=true` : Verhindert, dass Gazebo den Joint für Simulation optimiert

Verwendet von: robot.urdf.xacro (immer eingebunden)

Abhängigkeit: Externes Paket `nerf_launch_system` muss installiert sein

robot_core_bak.xml

Pfad: description/robot_core_bak.xml

Was es ist: Backup/Archiv-Datei der ursprünglichen `robot_core` -Version (plain URDF, kein Xacro). Wird **nicht** eingebunden und ist nur zur Referenz vorhanden. Gemäß Projekt-Konvention: auskommentierter/archivierter Code bleibt bis zur expliziten Freigabe.

Verwendet von: Nirgends (archiviert)

launch/ – Launch-Dateien

rsp.launch.py (*Robot State Publisher*)

Pfad: launch/rsp.launch.py

Was es ist: Basislaunch-Datei. Verarbeitet `robot.urdf.xacro` mit `xacro` zur Laufzeit und startet den `robot_state_publisher` -Node.

Was gestartet wird:

Node	Paket	Funktion
robot_state_publisher	robot_state_publisher	Publiziert /robot_description + TF-Baum

Argumente:

Argument	Standard	Beschreibung
use_sim_time	false	Simulationszeit aktivieren
use_ros2_control	true	ros2_control-Block in URDF einbinden
integrated_mode	false	Nerf + Basis als ein System
use_nerf_hardware	true	Nerf-Hardware-Interface einbinden
use_gazebo_classic	false	Gazebo Classic statt Ignition

Publizierte Topics:

- /robot_description (std_msgs/String)
- /tf und /tf_static (alle fixen Joints)

Verwendet von:

- launch_robot.launch.py
- launch_sim.launch.py
- gz_classic_launch_sim.launch.py

launch_robot.launch.py *(Echte Hardware – Basis)*

Pfad: launch/launch_robot.launch.py

Was es ist: Hauptlaunch für den echten Roboter-Antrieb. Startet Robot State Publisher, Joystick, Twist-Mux, Controller Manager und alle ros2_control-Spawner in **strikter Sequenz**, um DDS-Überlastung zu vermeiden.

Start-Sequenz (OnProcessExit-Chain):

```
[3s Verzögerung] controller_manager
└─(OnProcessStart)→ diff_drive_spawner
    └─(OnProcessExit)→ joint_broad_spawner
        └─(OnProcessExit)→ [wenn use_nerf_hardware=true]:
            trigger_controller
            └─ flywheel_controller
                └─ pusher_controller
                    └─ arming_controller
                        └─ nerf_control_node
```

Gestartete Komponenten:

Komponente	Typ	Beschreibung
rsp.launch.py	Include	URDF + TF publizieren
joystick.launch.py	Include	Joystick-Eingabe (joy_node extern)

twist_mux	Node	Multiplexer: Joystick > Tracker > Keyboard
ros2_control_node	Node	Controller Manager
diff_cont Spawner	Node	Differential-Drive-Controller laden
joint_broad Spawner	Node	Joint-State-Broadcaster laden
trigger_controller	Node (cond.)	Nerf Trigger (Servo)
flywheel_controller	Node (cond.)	Nerf Flywheels
pusher_controller	Node (cond.)	Dart Pusher
arming_controller	Node (cond.)	Sicherheitsarretierung
nerf_control_node	Node (cond.)	High-Level Nerf-Steuerung

Argumente:

Argument	Standard	Beschreibung
use_nerf_hardware	false	Nerf-Launcher aktivieren
auto_arm	false	Beim Start automatisch scharf schalten

Verwendet von: `launch_all_real.launch.py`

`launch_all_real.launch.py` *(Echte Hardware – Vollständig)*

Pfad: `launch/launch_all_real.launch.py`

Was es ist: Der **primäre Einstiegspunkt** für den echten Roboter. Kombiniert alle Subsysteme und ermöglicht per Argument die optionale Aktivierung von Lidar, Kamera und Gesichtserkennung.

Gestartete Komponenten:

Komponente	Bedingung	Beschreibung
<code>launch_robot.launch.py</code>	immer	Antrieb + Nerf
<code>rplidar.launch.py</code>	<code>launch_lidar=true</code>	RPLidar Scanner
<code>camera.launch.py</code>	<code>launch_camera=true + camera_type=v4l2</code>	v4l2-Kameratreiber
<code>real_camera.launch.py</code>	<code>launch_camera=true + camera_type=usb_cam</code>	usb_cam-Treiber (MJPEG)
<code>ball_tracker/face_tracker.launch.py</code>	<code>launch_face_tracker=true</code>	Gesichtserkennung (12s Verzögerung)

Argumente:

Argument	Standard	Beschreibung
----------	----------	--------------

launch_lidar	false	RPLidar starten
launch_camera	false	Kamera starten
camera_type	v4l2	Kameratreiber: v4l2 oder usb_cam
launch_face_tracker	false	Gesichtserkennung starten
target_person	``	Zielperson für Tracking
allow_search	false	Roboter dreht sich auf Suche
auto_arm	false	Nerf auto scharf schalten

Typische Verwendung:

```
ros2 launch gubot_one launch_all_real.launch.py
ros2 launch gubot_one launch_all_real.launch.py launch_camera:=true camera_type:=usb_cam
launch_face_tracker:=true
```

launch_sim.launch.py (Ignition Gazebo Simulation)

Pfad: launch/launch_sim.launch.py

Was es ist: Vollständige Simulationsumgebung mit **Ignition Gazebo** (ros_gz_sim). Startet Gazebo, spawnt den Roboter, richtet den ROS→Gazebo-Bridge ein und lädt alle Controller.

Gestartete Komponenten:

Komponente	Paket/Executable	Beschreibung
rsp.launch.py	Include	URDF + TF
joystick.launch.py	Include	Joystick
twist_mux	Node	Velocity-Multiplexer
gz_sim.launch.py	ros_gz_sim	Ignition Gazebo starten
ros_gz_sim create	Node	Roboter in Welt spawnen
parameter_bridge	ros_gz_bridge	ROS→Ignition Topic-Bridge
rviz2	Node	Visualisierung (view_bot.rviz)
Controller-Spawner	Nach spawn_entity	diff_cont, joint_broad, imu_broadcaster, alle Nerf-Controller

Bridge-Topics:

Topic	Richtung	Typ
/clock	GZ→ROS	roscpp_msgs/Clock
/scan	GZ→ROS	sensor_msgs/LaserScan

/camera/image_raw	GZ→ROS	sensor_msgs/Image
/camera/camera_info	GZ→ROS	sensor_msgs/CameraInfo
/imu_sensor/imu_data	GZ→ROS	sensor_msgs/Imu

Umgebungsvariablen (automatisch gesetzt):

- `IGN_GAZEBO_RESOURCE_PATH` → `~/ .gazebo/models`
- `MESA_GL_VERSION_OVERRIDE=4.5` + `MESA_GLSL_VERSION_OVERRIDE=450` (für Software-Rendering)
- `GZ_TRANSPORT_RCVHWM=1000`

Argumente:

Argument	Standard	Beschreibung
world	worlds/obstacles.world	Welt-Datei
use_sim_time	true	Simulationszeit

`gz_classic_launch_sim.launch.py` (*Gazebo Classic Simulation*)

Pfad: `launch/gz_classic_launch_sim.launch.py`

Was es ist: Wie `launch_sim.launch.py`, aber für **Gazebo Classic** (`gazebo_ros`). Nutzt `gazebo_ros spawn_entity.py` statt `ros_gz_sim create`, kein separater Bridge-Node nötig (Gazebo Classic publiziert direkt auf ROS2).

Unterschied zu `launch_sim.launch.py`:

Eigenschaft	<code>launch_sim</code> (Ignition)	<code>gz_classic_launch_sim</code> (Classic)
Gazebo-Paket	<code>ros_gz_sim</code>	<code>gazebo_ros</code>
Spawn-Executable	<code>ros_gz_sim create</code>	<code>gazebo_ros spawn_entity.py</code>
Bridge-Node	Ja (5 Topics)	Nein (integriert)
Standard-Welt	<code>obstacles.world</code>	<code>obstacles_classic.world</code>
RSP-Arg	<code>use_gazebo_classic=false</code>	<code>use_gazebo_classic=true</code>

Argumente:

Argument	Standard	Beschreibung
world	worlds/obstacles_classic.world	Welt-Datei
use_sim_time	true	Simulationszeit

`joystick.launch.py`

Pfad: `launch/joystick.launch.py`

Was es ist: Startet die Joystick-Steuerung. Kann den `joy_node` optional lokal starten oder annehmen, dass er auf einem anderen Gerät läuft (z. B. Laptop mit Controller, während der Roboter selbst keine direkte USB-Verbindung hat).

Gestartete Nodes:

Node	Paket	Conditional	Beschreibung
joy_node	joy	launch_joy_node=true	Liest /dev/input/js0
teleop_node	teleop_twist_joy	immer	Joy → Twist (/cmd_vel_joy)
nerf_joy	gubot_one	immer	Nerf-Steuerung via Controller

Topic-Flow:

```
/dev/input/js0 → joy_node → /joy → teleop_node → /cmd_vel_joy → twist_mux
                                     → nerf_joy → Nerf-Topics
```

Argumente:

Argument	Standard	Beschreibung
use_sim_time	false	Simulationszeit
launch_joy_node	true	joy_node lokal starten

Konfiguration: `config/joystick.yaml` (Achsenmapping, Deadband etc.)

Verwendet von: `launch_robot.launch.py` , `launch_sim.launch.py` , `gz_classic_launch_sim.launch.py`

camera.launch.py (v4l2 Kameratreiber)

Pfad: `launch/camera.launch.py`

Was es ist: Startet den `v4l2_camera_node` für die USB-Kamera am echten Roboter.

Konfiguration:

Parameter	Wert	Beschreibung
video_device	/dev/video0	USB-Kamera-Device
pixel_format	YUYV	Unkomprimiert, stabiler als MJPEG
image_size	[320, 240]	Reduziert für WLAN/Tailscale
time_per_frame	[1, 10]	10 FPS
camera_frame_id	camera_link_optical	TF-Frame (REP-103)

Publizierte Topics:

- `/camera/image_raw`
- `/camera/camera_info`

Verwendet von: `launch_all_real.launch.py` (wenn `camera_type=v4l2`)

`real_camera.launch.py` (*usb_cam Kameratreiber*)

Pfad: `launch/real_camera.launch.py`

Was es ist: Alternative zum `v4l2_camera` -Treiber. Nutzt `usb_cam` mit MJPEG-Dekodierung, optimiert für höhere Übertragungsrate über WLAN.

Konfiguration:

Parameter	Wert	Beschreibung
<code>video_device</code>	<code>/dev/video0</code>	USB-Kamera-Device
<code>pixel_format</code>	<code>mjpeg2rgb</code>	MJPEG dekodiert zu RGB
<code>output_encoding</code>	<code>bgr8</code>	OpenCV-kompatibel
<code>image_width/height</code>	<code>320x240</code>	Reduziert für WLAN
<code>framerate</code>	<code>10.0</code>	10 FPS
<code>camera_name</code>	<code>real_cam</code>	Für Kalibrierungsdatei

Publizierte Topics:

- `/camera/image_raw`
- `/camera/camera_info`

Kalibrierungsdatei: `config/camera_info/real_cam.yaml`

Verwendet von: `launch_all_real.launch.py` (wenn `camera_type=usb_cam`)

`rplidar.launch.py` (*RPLidar Scanner*)

Pfad: `launch/rplidar.launch.py`

Was es ist: Startet den RPLidar-Treiber für den Laser-Scanner. Hardcodierter USB-Pfad für den spezifischen Raspberry Pi USB-Hub-Port.

Konfiguration:

Parameter	Wert	Beschreibung
<code>serial_port</code>	<code>/dev/serial/by-path/platform-fd500000...</code>	Fester USB-Hub-Port
<code>frame_id</code>	<code>laser_frame</code>	TF-Frame (muss mit URDF übereinstimmen)
<code>angle_compensate</code>	<code>true</code>	Winkelkompensation aktiviert
<code>scan_mode</code>	<code>Standard</code>	Standard-Scanmodus

Publiziertes Topic: `/scan` (`sensor_msgs/LaserScan`)

Verwendet von: `launch_all_real.launch.py` (wenn `launch_lidar=true`)

navigation_launch.py (Nav2 Navigation Stack)

Pfad: launch/navigation_launch.py

Was es ist: Startet den vollständigen Nav2-Navigations-Stack für autonome Navigation. Alle Nodes werden als **Lifecycle-Nodes** gestartet und vom `lifecycle_manager` verwaltet.

Gestartete Nodes:

Node	Paket	Funktion
controller_server	nav2_controller	Folgt geplantem Pfad (z.B. DWB)
planner_server	nav2_planner	Berechnet globalen Pfad (z.B. NavFn)
recoveries_server	nav2_recoveries	Führt Recovery-Behaviors aus (Spin, Backup)
bt_navigator	nav2_bt_navigator	Koordiniert via Behavior Tree XML
waypoint_follower	nav2_waypoint_follower	Folgt Waypoint-Listen
lifecycle_manager_navigation	nav2_lifecycle_manager	Verwaltet alle obigen Nodes

Argumente:

Argument	Standard	Beschreibung
use_sim_time	false	Simulationszeit
autostart	true	Lifecycle-Nodes automatisch aktivieren
params_file	config/nav2_params.yaml	Nav2-Konfiguration
default_bt_xml_filename	nav2 Standard-BT	Behavior Tree XML
map_subscribe_transient_local	false	QoS für Map-Topic

localization_launch.py (AMCL Lokalisierung)

Pfad: launch/localization_launch.py

Was es ist: Startet `map_server` und AMCL (Adaptive Monte Carlo Localization) für die Positionsschätzung in einer bereits bekannten Karte.

Gestartete Nodes:

Node	Paket	Funktion
map_server	nav2_map_server	Lädt und publiziert Karte auf /map
amcl	nav2_amcl	Partikelfilter-Lokalisierung
lifecycle_manager_localization	nav2_lifecycle_manager	Lifecycle-Verwaltung

Argumente:

Argument	Standard	Beschreibung
map	maps/turtlebot3_world.yaml	Zu ladende Karte
use_sim_time	false	Simulationszeit
params_file	config/nav2_params.yaml	Nav2-Parameter

online_async_launch.py (SLAM Toolbox)

Pfad: launch/online_async_launch.py

Was es ist: Startet den `slam_toolbox` -Node im asynchronen Online-Modus – erstellt gleichzeitig eine Karte und lokalisiert den Roboter darin (kein vorab bekannte Karte nötig).

Gestarteter Node:

Node	Paket	Funktion
slam_toolbox	slam_toolbox	Async Online SLAM

Konfiguration: config/mapper_params_online_async.yaml

Intelligenz: Prüft automatisch, ob die übergebene `params_file` `slam_toolbox`-Parameter enthält; fällt sonst auf `default_params_file` zurück.

Argumente:

Argument	Standard	Beschreibung
use_sim_time	true	Simulationszeit
params_file	config/mapper_params_online_async.yaml	SLAM-Parameter

ball_tracker.launch.py (Ball-Tracker)

Pfad: launch/ball_tracker.launch.py

Was es ist: Wrapper-Launch für den externen `ball_tracker` -Package. Wählt je nach `sim_mode` die passende Parameterdatei (Simulation vs. echter Roboter).

Konfiguration:

sim_mode	Parameterdatei
true	config/ball_tracker_params_sim.yaml
false	config/ball_tracker_params_robot.yaml

Feste Argumente an `ball_tracker.launch.py` :

- `image_topic` : /camera/image_raw
- `cmd_vel_topic` : /cmd_vel_tracker
- `enable_3d_tracker` : true

Argumente:

Argument	Standard	Beschreibung
sim_mode	false	Simulations- oder Roboter-Parameter

Abhängigkeitsmatrix

Welche Launch-Datei wird von wem aufgerufen?

```
launch_all_real.launch.py
├─ launch_robot.launch.py
│   └─ rsp.launch.py
│       └─ joystick.launch.py
├─ rplidar.launch.py          (optional)
├─ camera.launch.py          (optional, v4l2)
├─ real_camera.launch.py      (optional, usb_cam)
└─ ball_tracker/face_tracker.launch.py (optional)

launch_sim.launch.py (Ignition)
├─ rsp.launch.py
└─ joystick.launch.py

gz_classic_launch_sim.launch.py (Gazebo Classic)
├─ rsp.launch.py
└─ joystick.launch.py
```

Welche URDF-Dateien werden von wem eingebunden?

```
robot.urdf.xacro ← rsp.launch.py, launch_robot.launch.py
├─ robot_core.xacro
│   └─ inertial_macros.xacro
├─ ros2_control.xacro          (use_ros2_control=true, use_gazebo_classic=false)
├─ gz_classic_ros2_control.xacro (use_ros2_control=true, use_gazebo_classic=true)
├─ gazebo_control.xacro        (use_ros2_control=false)
├─ camera.xacro
├─ face.xacro
├─ nerf_launcher.xacro
│   └─ nerf_launch_system/description/urdf/launcher.urdf.xacro
├─ [lidar.xacro]                (auskommentiert)
└─ [depth_camera.xacro]         (auskommentiert)
```

Externe Paket-Abhängigkeiten

Paket	Verwendet in
robot_state_publisher	rsp.launch.py
controller_manager	launch_robot.launch.py, launch_sim.launch.py
twist_mux	launch_robot.launch.py, launch_sim.launch.py

joy + teleop_twist_joy	joystick.launch.py
ros_gz_sim + ros_gz_bridge	launch_sim.launch.py
gazebo_ros	gz_classic_launch_sim.launch.py
v4l2_camera	camera.launch.py
usb_cam	real_camera.launch.py
rplidar_ros	rplidar.launch.py
slam_toolbox	online_async_launch.py
nav2_*	navigation_launch.py, localization_launch.py
nerf_launch_system	nerf_launcher.xacro, launch_robot.launch.py
ball_tracker	ball_tracker.launch.py, launch_all_real.launch.py
diffdrive_arduino	ros2_control.xacro, gz_classic_ros2_control.xacro